



PYTHON

6-MONTH JOB-READY ROADMAP

From Zero to Employed Developer

Complete curriculum • Real-world projects • Job interview prep
No prior coding experience required

6 Months
Duration

30+ Projects
Build Portfolio

7 Domains
Specializations

Job Ready
Goal

How to Use This Roadmap

This roadmap is designed for complete beginners. Follow the months in order — each month builds on the previous one. Do not skip topics. The projects are as important as the theory, so build every single one.

Daily Study Schedule

Topic / Module	What You'll Learn	Est. Time
Weekdays (Mon–Fri)	1–2 hours of learning theory & watching tutorials 30–60 minutes of coding practice (solve problems) - Write code every day, even if only 20 lines	2–3 hrs/day
Weekends (Sat–Sun)	3–4 hours dedicated to project building - Review the week's concepts - Push code to GitHub	3–4 hrs/day

Tools to Install Before You Start

Topic / Module	What You'll Learn	Est. Time
Python Installer	- Download Python 3.11+ from python.org - Check 'Add Python to PATH' during installation - Verify install: open terminal, type <code>python --version</code>	15 min
VS Code (Code Editor)	- Download from code.visualstudio.com - Install Python extension by Microsoft - Install Pylance and GitLens extensions	20 min
Git & GitHub	- Download Git from git-scm.com - Create a free account on github.com - This is where your portfolio will live	30 min
Terminal / Command Prompt	- Windows: use PowerShell or Windows Terminal - Mac/Linux: use built-in Terminal - Learn to navigate folders with <code>cd</code>, <code>ls</code>, <code>mkdir</code> commands	15 min

MONTH 1
Weeks 1–4**Python Fundamentals — Learning the Language**

Goal: Understand Python syntax, write basic programs, and build your first mini-projects. By end of month 1 you should be able to write programs that take input, make decisions, loop, and work with data.

Week 1 – Python Basics

Topic / Module	What You'll Learn	Est. Time
Setting Up Environment	- Install Python + VS Code - Run your first program: <code>print('Hello World!')</code> - Understand the REPL (interactive shell) - Learn how to run <code>.py</code> files from terminal	2 days
Variables & Data Types	<code>int</code> (whole numbers: 5, 100, -3) <code>float</code> (decimal numbers: 3.14, -0.5) <code>str</code> (text: 'Hello', 'Python') <code>bool</code> (True / False) <code>type()</code> function to check data type - Variable naming rules: no spaces, no numbers at start	2 days
Basic Input & Output	<code>print()</code> — display output to screen <code>input()</code> — take input from user - f-strings for formatting: <code>f'Hello {name}'</code> - String concatenation and repetition	1 day

Week 2 – Control Flow

Topic / Module	What You'll Learn	Est. Time
Operators	- Arithmetic: <code>+</code>, <code>-</code>, <code>*</code>, <code>/</code>, <code>//</code>, <code>%</code>, <code>**</code> - Comparison: <code>==</code>, <code>!=</code>, <code>></code>, <code><</code>, <code>>=</code>, <code><=</code> - Logical: <code>and</code>, <code>or</code>, <code>not</code> - Assignment: <code>=</code>, <code>+=</code>, <code>-=</code>, <code>*=</code>	1 day
Conditional Statements	<code>if</code> statement — execute code if condition is True <code>elif</code> — check another condition <code>else</code> — catch-all when all conditions are False - Nested <code>if</code> statements - Ternary expression: <code>x if condition else y</code>	2 days
Loops	<code>for</code> loop — iterate over a sequence <code>while</code> loop — repeat until condition is False <code>range()</code> function: <code>range(10)</code>, <code>range(1,11)</code>, <code>range(0,20,2)</code>	2 days

- `break` — exit loop early
- `continue` — skip to next iteration
- Nested loops (loop inside a loop)

Week 3 – Data Structures

Topic / Module	What You'll Learn	Est. Time
Lists	• Create: <code>[1, 2, 3]</code> or <code>list()</code> • Access by index: <code>my_list[0]</code>, <code>my_list[-1]</code> • Slicing: <code>my_list[1:4]</code> • Methods: <code>append</code>, <code>remove</code>, <code>pop</code>, <code>sort</code>, <code>reverse</code>, <code>len</code> • List comprehension: <code>[x*2 for x in numbers]</code>	2 days
Tuples & Sets	• Tuple: immutable list — <code>(1, 2, 3)</code> • When to use tuples vs lists • Set: no duplicates — <code>{1, 2, 3}</code> • Set operations: union, intersection, difference	1 day
Dictionaries	• Key-value pairs: <code>{'name': 'Ali', 'age': 25}</code> • Access values: <code>d['key']</code> or <code>d.get('key')</code> • Add/update/delete keys • Methods: <code>keys()</code>, <code>values()</code>, <code>items()</code> • Looping through dictionaries • Nested dictionaries	2 days

Week 4 – Functions & Modules

Topic / Module	What You'll Learn	Est. Time
Functions	• Define with <code>def</code> keyword • Parameters and arguments • Return values with <code>return</code> • Default parameter values • Keyword arguments vs positional arguments • <code>*args</code> and <code>**kwargs</code> (variable arguments) • Lambda functions: <code>lambda x: x * 2</code> • Scope: local vs global variables	3 days
Built-in Functions & Modules	• Built-ins: <code>len</code>, <code>range</code>, <code>zip</code>, <code>map</code>, <code>filter</code>, <code>enumerate</code>, <code>sorted</code> • <code>math</code> module: <code>sqrt</code>, <code>ceil</code>, <code>floor</code>, <code>pi</code> • <code>random</code> module: <code>random()</code>, <code>randint()</code>, <code>choice()</code> • Import syntax: <code>import</code>, <code>from...import</code>, <code>as</code> (alias) • Creating your own module (<code>.py</code> file)	2 days

Month 1 Projects to Build

 Calculator App Build a command-line calculator that takes two numbers and an operator (+, -, *, /) from the user and returns the result. <i>Skills: Variables, input/output, conditionals, functions</i>	Beginner
 Number Guessing Game Computer picks a random number 1-100. User guesses. Program says 'Too High' or 'Too Low' until correct. Track number of guesses. <i>Skills: Loops, conditionals, random module, input</i>	Beginner
 Simple Contact Book Store contacts (name, phone, email) in a dictionary. Allow user to add, search, and delete contacts via a menu. <i>Skills: Dictionaries, lists, functions, loops</i>	Beginner

MONTH 2
Weeks 5–8**Object-Oriented Programming & File Handling**

Goal: Write organized, reusable, professional code using classes and objects. Learn to read/write files, handle errors, and work with libraries.

Week 5 – Object-Oriented Programming (OOP)

Topic / Module	What You'll Learn	Est. Time
Classes & Objects	- What is a class? Blueprint for objects - Creating a class with class keyword <code>__init__</code> method — constructor - Instance variables (<code>self.name = name</code>) - Creating objects (instances) from a class - Accessing attributes and methods	2 days
OOP Principles	- Encapsulation — hide internal data - Inheritance — child class inherits from parent - Polymorphism — same method, different behavior <code>super()</code> — call parent class methods - Method overriding - Class variables vs instance variables	3 days

Week 6 – Error Handling & Exceptions

Topic / Module	What You'll Learn	Est. Time
Try/Except Blocks	<code>try</code> — code that might fail <code>except</code> — handle the error <code>else</code> — runs if no error occurred <code>finally</code> — always runs (cleanup) - Multiple except blocks for different errors - Catching specific exceptions: <code>ValueError</code>, <code>TypeError</code>, <code>FileNotFoundError</code>	2 days
Custom Exceptions	- Creating your own exception classes <code>raise</code> keyword — throw exceptions manually <code>assert</code> statement for debugging - Logging errors with the logging module	2 days
Debugging Techniques	- Using <code>print()</code> for debugging - VS Code debugger — breakpoints, step through - Reading error tracebacks - Common errors: <code>NameError</code>, <code>IndexError</code>, <code>KeyError</code>	1 day

Week 7 – File Handling

Topic / Module	What You'll Learn	Est. Time
Reading & Writing Files	• <code>open()</code> function with modes: <code>r</code>, <code>w</code>, <code>a</code>, <code>rb</code>, <code>wb</code> • <code>read()</code>, <code>readline()</code>, <code>readlines()</code> • <code>write()</code> and <code>writelines()</code> • <code>with</code> statement (context manager) — auto-closes file • Working with file paths	2 days
CSV & JSON Files	• <code>csv</code> module: read/write spreadsheet-style data • <code>json</code> module: <code>dumps()</code>, <code>loads()</code>, <code>dump()</code>, <code>load()</code> • Reading JSON APIs (preview) • Working with real data files	2 days
OS Module	• <code>os.path</code> — check if file/folder exists • <code>os.listdir()</code> — list files in directory • <code>os.makedirs()</code> — create folders • <code>os.rename()</code>, <code>os.remove()</code> — file management • <code>pathlib</code> module (modern alternative)	1 day

Week 8 – Python Standard Library Deep Dive

Topic / Module	What You'll Learn	Est. Time
datetime Module	• Get current date/time: <code>datetime.now()</code> • Date arithmetic (add/subtract days) • Format dates: <code>strftime()</code> • Parse strings to dates: <code>strptime()</code>	1 day
collections Module	• <code>Counter</code> — count occurrences • <code>defaultdict</code> — dict with default values • <code>deque</code> — fast append/pop from both ends • <code>namedtuple</code> — tuple with named fields	1 day
itertools & functools	• <code>itertools</code>: <code>chain</code>, <code>product</code>, <code>combinations</code>, <code>permutations</code> • <code>functools</code>: <code>reduce</code>, <code>partial</code>, <code>lru_cache</code> • When and why to use these tools	1 day
Regular Expressions (re)	• Pattern matching: <code>re.match()</code>, <code>re.search()</code>, <code>re.findall()</code> • Common patterns: email, phone, URL validation • Groups and substitution: <code>re.sub()</code> • Practical use: form validation	2 days

Month 2 Projects to Build

 Bank Account System Create a BankAccount class with methods: deposit, withdraw, check_balance. Include error handling for overdraft and invalid amounts. Save transactions to a file. <i>Skills: OOP, file handling, exceptions</i>	Intermediate
 Student Grade Manager OOP-based system with Student class. Read student data from CSV, calculate grades/GPA, sort students, export report to a new CSV. <i>Skills: OOP, CSV files, sorting, exceptions</i>	Intermediate
 Text File Analyzer Read a .txt file and count: total words, unique words, most common 10 words, number of sentences, average word length. Display formatted report. <i>Skills: File I/O, dictionaries, strings, collections</i>	Beginner

MONTH 3
Weeks 9–12**Web Development with Flask + Databases**

Goal: Build real websites and web applications. Learn backend development, databases, and how the internet works.

Week 9 – Web Fundamentals & Flask Basics

Topic / Module	What You'll Learn	Est. Time
How the Web Works	- HTTP protocol: GET vs POST requests - Client (browser) vs Server (your Python code) - URLs, endpoints, status codes (200, 404, 500) - HTML basics (just enough to build pages) - JSON as data format for APIs	1 day
Flask Setup & First App	- Install Flask: <code>pip install flask</code> - Virtual environments: <code>python -m venv venv</code> <code>requirements.txt</code> — track dependencies - First Flask app: <code>app.py</code> with <code>@app.route('/')</code> - Debug mode and auto-reload	2 days
Flask Routing & Templates	- URL routing with <code>@app.route()</code> - Dynamic URLs: <code>/user/<username></code> - Jinja2 templates — HTML with Python logic - Template inheritance: <code>base.html</code> + child pages - Passing data from Python to HTML templates - Static files: CSS, JavaScript, images	2 days

Week 10 – Forms, Sessions & Authentication

Topic / Module	What You'll Learn	Est. Time
HTML Forms & Flask Forms	- HTML form elements: input, select, textarea - Handling POST requests in Flask - WTForms library for form validation - CSRF protection - Flash messages (success/error notifications)	2 days
User Authentication	- Sessions and cookies - User registration (hash passwords with <code>bcrypt</code>) - Login and logout functionality - Flask-Login library - Protecting routes with <code>@login_required</code> - Remember me feature	3 days

Week 11 – Databases with SQLite & SQLAlchemy

Topic / Module	What You'll Learn	Est. Time
SQL Basics	- What is a database? Tables, rows, columns - CREATE TABLE, INSERT, SELECT, UPDATE, DELETE - WHERE clause for filtering - ORDER BY, LIMIT - JOINS to connect related tables - Primary keys and foreign keys	2 days
SQLAlchemy ORM	- Install: pip install flask-sqlalchemy - Define models as Python classes - db.create_all() — create tables - CRUD operations: add, query, update, delete - Relationships: one-to-many, many-to-many - Database migrations with Flask-Migrate	3 days

Week 12 – REST APIs

Topic / Module	What You'll Learn	Est. Time
Building a REST API	- What is an API? Why REST? - JSON responses with jsonify() - HTTP methods: GET, POST, PUT, DELETE - API endpoints and resources - Status codes: 200, 201, 400, 401, 404, 500 - Postman tool for testing APIs	3 days
API Authentication	- API keys - JWT tokens (JSON Web Tokens) - Flask-JWT-Extended library - Protecting API endpoints	2 days

Month 3 Projects to Build

 Personal Blog Website Full website: user registration/login, create/edit/delete blog posts, display all posts on homepage. Use SQLAlchemy for database storage. <i>Skills: Flask, SQLAlchemy, authentication, templates</i>	Intermediate
 Task Manager REST API Build a REST API for a to-do app. Endpoints: GET all tasks, POST new task, PUT update task, DELETE task. Include JWT authentication. <i>Skills: Flask REST API, JWT, SQLAlchemy, Postman</i>	Intermediate

URL Shortener

Users paste a long URL, get a short link. Redirect short link to original. Track number of clicks per link. Admin page to view stats.

Skills: Flask, SQLite, routing, database

Intermediate

MONTH 4
Weeks 13–16**Data Science & Automation**

Goal: Work with real data, automate repetitive tasks, and build data pipelines. These skills are in massive demand across all industries.

Week 13 – NumPy & Pandas

Topic / Module	What You'll Learn	Est. Time
NumPy	- Install: <code>pip install numpy</code> - Arrays vs Python lists (faster, less memory) - Creating arrays: <code>np.array()</code>, <code>np.zeros()</code>, <code>np.ones()</code>, <code>np.arange()</code> - Array operations: math, slicing, reshaping - Broadcasting: operate on arrays of different sizes <code>np.mean()</code>, <code>np.sum()</code>, <code>np.max()</code>, <code>np.min()</code>	2 days
Pandas	- DataFrame — table-like data structure - Series — single column - Load data: <code>pd.read_csv()</code>, <code>read_excel()</code>, <code>read_json()</code> - Explore data: <code>df.head()</code>, <code>df.info()</code>, <code>df.describe()</code> - Select columns: <code>df['col']</code> or <code>df[['c1','c2']]</code> - Filter rows: <code>df[df['age'] > 25]</code> - Handle missing data: <code>dropna()</code>, <code>fillna()</code> - Group data: <code>df.groupby('city').mean()</code> - Merge DataFrames: <code>merge()</code>, <code>concat()</code>	3 days

Week 14 – Data Visualization

Topic / Module	What You'll Learn	Est. Time
Matplotlib	- Line plots, bar charts, scatter plots, histograms - Customize: titles, labels, colors, grid - Multiple plots: subplots - Save charts as PNG/PDF: <code>plt.savefig()</code>	2 days
Seaborn	- Beautiful statistical visualizations - heatmap, pairplot, boxplot, violinplot - Distribution plots, count plots - Correlation matrices	1 day
Plotly (Interactive Charts)	- Interactive charts users can zoom/hover - Line, bar, pie, scatter, map charts <code>plotly.express</code> — quick charts - Export to HTML for embedding in websites	2 days

Week 15 – Web Scraping

Topic / Module	What You'll Learn	Est. Time
requests & BeautifulSoup	- requests.get(url) — fetch web pages - Parse HTML with BeautifulSoup - Find elements: find(), find_all(), select() - Extract text, links, images, tables - Handle headers and cookies - Respect robots.txt and terms of service	2 days
Selenium (Dynamic Sites)	- Install Selenium + ChromeDriver - Automate browser: open URLs, click buttons, fill forms - Wait for elements to load - Scrape JavaScript-rendered content - Taking screenshots programmatically	3 days

Week 16 – Automation & Scripting

Topic / Module	What You'll Learn	Est. Time
File & System Automation	- Bulk rename, move, copy, delete files - Watch folders for new files - Schedule tasks: schedule library - Send automated emails: smtplib - Generate PDF reports: reportlab or fpdf	2 days
Excel Automation	- Read Excel: openpyxl or xlrd - Write/format Excel: openpyxl - Create charts in Excel with Python - Automate repetitive spreadsheet tasks	2 days
API Automation	- Automate data fetching from public APIs - OpenWeather, CoinGecko, News API - Schedule API calls and store results - Build automated reports with fetched data	1 day

Month 4 Projects to Build

 Data Analysis Dashboard Analyze a real dataset (COVID-19, Netflix, Titanic from Kaggle). Clean data, answer 5 analysis questions, create 5 visualizations. Export to HTML dashboard. <i>Skills: Pandas, Matplotlib, Seaborn, Plotly</i>	Intermediate
--	---------------------

Job Listings Scraper

Scrape job listings from a job board (title, company, location, salary). Store in CSV/Excel. Auto-run daily with scheduler. Email yourself new listings.

Skills: BeautifulSoup, Selenium, Pandas, schedule, smtplib

Intermediate

Excel Report Automator

Read sales data from Excel, calculate monthly totals, identify top performers, create formatted Excel report with charts automatically.

Skills: openpyxl, Pandas, automation

Intermediate

MONTH 5
Weeks 17–20**Advanced Python + Choose Your Specialization**

Goal: Write professional-grade Python. Learn advanced concepts used in real companies. Then pick ONE specialization to go deep.

Week 17 – Advanced Python Concepts

Topic / Module	What You'll Learn	Est. Time
Decorators	• Functions as first-class objects • Closures and higher-order functions • Writing your own decorators with @ • Common use cases: timing, logging, authentication • functools.wraps to preserve function metadata	2 days
Generators & Iterators	• yield keyword — create generators • Memory-efficient iteration over large data • Generator expressions vs list comprehensions • Custom iterators with __iter__ and __next__ • itertools for advanced iteration	1 day
Context Managers	• with statement — how it works • Creating context managers with __enter__ / __exit__ • contextlib module • Real-world use: database connections, file locks	1 day
Concurrency	• threading module — run tasks in parallel • multiprocessing — use multiple CPU cores • asyncio — asynchronous programming with async/await • When to use which approach • aiohttp for async HTTP requests	2 days

Week 18 – Professional Development Practices

Topic / Module	What You'll Learn	Est. Time
Testing	• Why testing matters (saves hours of debugging) • unittest — Python built-in test framework • pytest — popular testing library • Write test functions: assert statements • Test coverage: what % of code is tested? • TDD (Test-Driven Development) basics	2 days
Git & Version Control	• git init, add, commit — save snapshots	2 days

	• git push, pull — sync with GitHub • Branches: git branch, checkout, merge • Resolving merge conflicts • Pull requests (collaboration workflow) • Writing good commit messages	
Clean Code & Best Practices	• PEP 8 — Python style guide • Type hints: <code>def greet(name: str) -> str:</code> • Docstrings for functions and classes • Code reviews — what to look for • Refactoring: improve code without changing behavior • Design patterns: Singleton, Factory, Observer	1 day

Weeks 19–20 – Choose Your Specialization

Pick ONE path below based on your job goal. Spend 2 full weeks going deep on it.

 Path A: Full-Stack Web Dev • Django framework (models, views, templates) • Django REST Framework for APIs • PostgreSQL database • Deploy with Heroku or Railway • Basic React frontend (optional)	 Path B: Machine Learning • scikit-learn: classification, regression • Data preprocessing pipelines • Model evaluation and metrics • TensorFlow or PyTorch basics • Deploy model as Flask API
 Path C: DevOps / Backend • FastAPI framework (faster than Flask) • Docker — containerize your apps • Linux command line proficiency • CI/CD pipelines with GitHub Actions • AWS/cloud basics (EC2, S3, Lambda)	 Path D: Automation / Scripting • Airflow for workflow automation • Selenium advanced patterns • Build CLI tools with argparse/click • Scrapy framework for large-scale scraping • Integrate with Slack, Gmail, Notion APIs

Month 5 Projects to Build

 E-Commerce API (Django/FastAPI) Full REST API: products, users, orders, payments. JWT auth, admin panel, Swagger docs. Deploy to cloud. <i>Skills: Django/FastAPI, PostgreSQL, JWT, deployment</i>	Advanced
 ML Price Predictor	Advanced

Train ML model on house/car price dataset. Web form for user input, model predicts price. Visualize feature importance.

Skills: scikit-learn, Flask, Pandas, Matplotlib



MONTH 6
Weeks 21–24

Portfolio, Job Prep & Real-World Deployment

Goal: Build your strongest portfolio project, prepare for technical interviews, polish your GitHub, and start applying for jobs.

Week 21 – Capstone Project

Build ONE impressive, fully deployed project. This is your centerpiece portfolio item. It must be live on the internet, have a GitHub repo with a good README, and solve a real problem.

Topic / Module	What You'll Learn	Est. Time
Web Dev Capstone Ideas	- SaaS tool: subscription-based web app - Social media clone (Twitter/Instagram) - E-learning platform with video/quizzes - Real-time chat app with Flask-SocketIO	1 week
Data/ML Capstone Ideas	- Sentiment analysis dashboard for tweets - Stock price predictor with news correlation - Recommendation engine (movies/products) - Image classifier with neural networks	1 week
Required for All Projects	- Live deployment (Heroku, Railway, Render, or AWS) - GitHub repo with detailed README.md - Demo video or screenshots - Documented API (Swagger or Postman collection)	Throughout

Week 22 – Technical Interview Prep

Topic / Module	What You'll Learn	Est. Time
Data Structures & Algorithms	- Arrays & Strings: two-pointer, sliding window - Linked Lists: traversal, reversal - Stacks & Queues: bracket matching, BFS - Trees & Graphs: DFS, BFS traversal - Hash Maps: frequency counting, anagrams - Sorting: bubble, merge, quick sort - Practice on: LeetCode, HackerRank, CodeWars (50+ problems)	5 days
Python Interview Questions	- How does Python manage memory? (garbage collection) - Mutable vs immutable objects	2 days

	• GIL (Global Interpreter Lock) explained • List vs tuple vs set vs dict — when to use each • Shallow vs deep copy • What are decorators and generators? • Explain OOP: inheritance, polymorphism, encapsulation	
--	--	--

Week 23 – System Design & Career Prep

Topic / Module	What You'll Learn	Est. Time
System Design Basics	• How to design a URL shortener • How to design a chat application • Databases: SQL vs NoSQL — when to use each • Caching strategies (Redis) • Load balancing concepts • Horizontal vs vertical scaling	3 days
Resume & LinkedIn	• Resume format for developers (1 page) • How to describe projects: problem → solution → impact • Quantify results: '50% faster', 'handles 1000 users' • LinkedIn profile: headline, about, skills, projects • GitHub profile README — make it stand out	2 days

Week 24 – Applying for Jobs

Topic / Module	What You'll Learn	Est. Time
Job Search Strategy	• LinkedIn Jobs — set up job alerts • Indeed, Glassdoor, Rozee.pk (Pakistan), Upwork • Naukri.com, Angellist for startups • Apply to 5-10 jobs per day consistently	Ongoing
The Interview Process	• HR screen: behavioral questions (STAR method) • Technical phone screen: coding on shared editor • Take-home assignment: build a small feature • Final technical + culture fit interview	Prepare always
Behavioral Questions to Prepare	• Tell me about yourself (2-minute pitch) • Why do you want to be a developer? • Describe a challenging bug you fixed • How do you learn new technologies? • Tell me about a project you're proud of	3 days

Freelancing as Alternative	• Upwork, Fiverr, Freelancer.com • Start with small gigs to build reviews • Common gigs: web scraping, automation scripts, Flask APIs, data analysis • Build a portfolio on your own website	Parallel
-----------------------------------	--	-----------------

Essential Learning Resources

Free Learning Platforms

Platform	Best For	Cost
Python.org Official Docs	Official reference for any Python feature	Free
freeCodeCamp (YouTube)	Full Python courses for beginners	Free
CS50P (Harvard)	Free Python course, certificates	Free
Real Python (realpython.com)	In-depth tutorials and articles	Mostly Free
Corey Schafer (YouTube)	Best Python tutorials on YouTube	Free
LeetCode	Algorithm practice (Easy/Medium problems)	Free tier
Kaggle	Datasets + free ML courses + notebooks	Free
The Odin Project	Web development curriculum	Free
Udemy	Paid courses (buy on sale for \$10–15)	Paid

Recommended Python Packages by Category

Topic / Module	What You'll Learn	Est. Time
Web Development	- Flask, Django, FastAPI, requests, httpx - SQLAlchemy, Flask-Login, WTForms, gunicorn - Celery (task queues), Redis, Stripe (payments)	
Data Science	- numpy, pandas, matplotlib, seaborn, plotly - scikit-learn, statsmodels, scipy - Jupyter Notebook / JupyterLab	
Machine Learning	- TensorFlow, PyTorch, Keras - XGBoost, LightGBM, CatBoost - Hugging Face transformers (NLP)	
Automation & Scraping	- selenium, playwright, scrapy, beautifulsoup4 - pyautogui, schedule, celery - openpyxl, python-docx, fpdf2	
Utilities & Productivity	- click, argparse (CLI tools) - python-dotenv (environment variables) - pydantic (data validation), loguru (logging) - black (code formatter), isort, pylint	

6-Month Progress Checklist

Print this page and check off items as you complete them. This is your accountability tracker.

MONTH 1 — Fundamentals

- ☐ Install Python, VS Code, and Git
- ☐ Print 'Hello World' and understand how to run .py files
- ☐ Understand all basic data types: int, float, str, bool
- ☐ Write programs using if/elif/else
- ☐ Use for and while loops comfortably
- ☐ Work with lists, tuples, sets, and dictionaries
- ☐ Write and call your own functions
- ☐ Import and use standard library modules
- ☐ Build: Calculator App
- ☐ Build: Number Guessing Game
- ☐ Build: Simple Contact Book
- ☐ Push all projects to GitHub

MONTH 2 — OOP & Files

- ☐ Create classes with __init__, attributes, and methods
- ☐ Implement inheritance and method overriding
- ☐ Handle exceptions with try/except
- ☐ Read and write text files
- ☐ Parse and write CSV and JSON files
- ☐ Use datetime, collections, re modules
- ☐ Build: Bank Account System (OOP)
- ☐ Build: Student Grade Manager (CSV)
- ☐ Build: Text File Analyzer

MONTH 3 — Web Development

- ☐ Build a basic Flask app with routes and templates
- ☐ Create HTML forms and handle POST requests
- ☐ Implement user registration and login
- ☐ Design and query a database with SQLAlchemy
- ☐ Build a REST API with GET/POST/PUT/DELETE endpoints
- ☐ Test API endpoints with Postman
- ☐ Build: Personal Blog Website
- ☐ Build: Task Manager REST API
- ☐ Build: URL Shortener

MONTH 4 — Data Science & Automation

- ☐ Manipulate arrays with NumPy
- ☐ Load, clean, and analyze data with Pandas
- ☐ Create 5+ types of charts with Matplotlib/Seaborn
- ☐ Scrape a real website with BeautifulSoup
- ☐ Automate a browser task with Selenium
- ☐ Automate a file management task
- ☐ Build: Data Analysis Dashboard
- ☐ Build: Job Listings Scraper
- ☐ Build: Excel Report Automator

MONTH 5 — Advanced Python & Specialization

- ☐ Write and use custom decorators
- ☐ Use generators for memory-efficient code
- ☐ Write unit tests with pytest
- ☐ Use Git branches and pull requests
- ☐ Complete 30+ LeetCode Easy/Medium problems
- ☐ Go deep on your chosen specialization (2 weeks)
- ☐ Build 1 advanced specialization project

MONTH 6 — Portfolio & Job Ready

- ☐ Deploy capstone project live on the internet
- ☐ Write a detailed GitHub README for capstone
- ☐ Complete 50+ LeetCode problems total
- ☐ Prepare answers to 10+ behavioral questions
- ☐ Update resume with all projects and skills
- ☐ Optimize LinkedIn profile
- ☐ Apply to at least 5 jobs per day
- ☐ Complete at least 3 mock technical interviews

Final Tips for Success

✓ DO These Things

- Code every single day, even 20 minutes
- Build projects — don't just watch tutorials
- Embrace errors — they are your teachers
- Ask for help on Stack Overflow, Reddit r/learnpython
- Join Python communities and Discord servers
- Teach what you learn (blog, tweet, explain to others)
- Track your daily progress in a journal

✗ AVOID These Mistakes

- Tutorial hell — watching without building
- Skipping projects to 'finish the course faster'
- Copying code without understanding it
- Waiting until you 'know enough' to start applying
- Learning multiple languages at once
- Ignoring fundamentals to jump to frameworks
- Giving up after 1-2 weeks of struggle

Remember: Every expert was once a beginner.

The only way to fail this roadmap is to stop. Keep going, keep building, and in 6 months you will look back amazed at how far you have come. You have everything you need — now start.